



Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»

ФАКУЛЬТЕТ «Программной инженерии и компьютерной техники»

ОБРАЗОВАТЕЛЬНАЯ ПРОГРАММА «Проектирование и разработка систем больших данных»

НАПРАВЛЕНИЕ ПОДГОТОВКИ (СПЕЦИАЛЬНОСТЬ) «09.04.04 Программная инженерия»

Программной инженерии и компьютерной техники (ФПИ и КТ)

ОТЧЕТ

по лабораторной работе №1

по курсу «Структуры и алгоритмы в базах данных и распределенных
системах»

на тему: «Реализация алгоритмов хэширования (perfect, extendible, min)»

Обучающийся: Постнов С. А., Р4135
(Ф.И.О, № группы)

Преподаватель Платонов А. В., доцент
(Ф.И.О, должность)

Преподаватель Портнов П. В., ассистент
(Ф.И.О, должность)

2026 г.

СОДЕРЖАНИЕ

1	Теоретическая часть	3
1.1	Perfect hashing	3
2	Практическая часть	4
2.1	Реализация perfect hash таблицы	4
3	Исследовательская часть	5
3.1	Методика замеров	5
3.2	Результаты	5
4	Вывод	7

1 Теоретическая часть

1.1 Perfect hashing

Задача: хранить пары ключ–значение так, чтобы операции `set/get` выполнялись за $O(1)$ (в среднем) и не возникало коллизий для фиксированного набора ключей.

Perfect hash для множества ключей S — это хэш–функция h , которая отображает ключи из S в индексы таблицы без коллизий. В классическом варианте (FKS) используется двухуровневая схема: первичная таблица распределяет ключи по корзинам, а для каждой корзины строится вторичная таблица, в которой подбираются параметры так, чтобы коллизий не было.

В данной работе реализована схожая идея:

- первичная таблица имеет размер, равный ближайшему простому $\geq n$;
- в ячейке хранится либо одна пара `key/value`, либо указатель на вторичную таблицу;
- при коллизии создаётся/перестраивается вторичная таблица: увеличивается её размер и перебираются различные хэш–функции до получения раскладки без коллизий.

2 Практическая часть

2.1 Реализация perfect hash таблицы

Публичный интерфейс реализован в файлах `inc/perfect.h` и `src/perfect.c`:

- `create_perfect_table` создаёт таблицу нужного размера;
- `perfect_set` добавляет/обновляет пару ключ–значение;
- `perfect_get` выполняет поиск значения по ключу;
- `free_perfect_table` освобождает память.

Структура `perfect_hash_table_t` содержит массив указателей на `ceil_value_t`. Каждая ячейка либо хранит одну пару `key/value`, либо указывает на вторичную таблицу (вложенный `perfect_hash_table_t`).

Для хэширования строк используются функции из `inc/hash.h`. При перестроении вторичных таблиц перебирается набор хэш–функций до построения раскладки без коллизий.

3 Исследовательская часть

3.1 Методика замеров

Для замеров использована программа `src/benchmark.c`. Для каждого датасета из `data/bench/manifest.txt`:

- 1) создаётся пустая perfect hash таблица;
- 2) измеряется время вставки всех n ключей (`perfect_set`);
- 3) измеряется время чтения всех n ключей (`perfect_get`).

Каждый прогон усредняется по `BENCH_ITERS` (по умолчанию 5).

3.2 Результаты

Таблица 3.1 содержит усреднённые значения задержек и пропускной способности.

Таблица 3.1 – Результаты бенчмарка perfect hash (среднее по итерациям)

N	вставка, нс/оп	вставка, млн оп/с	поиск, нс/оп	поиск, млн оп/с
50	344.0	2.98	80.0	12.50
250	528.0	1.90	112.0	8.95
1250	226.4	4.42	93.0	10.76
7500	314.9	3.24	82.8	12.19
20000	404.5	2.51	66.2	15.22
50000	217.0	4.61	52.4	19.09
100000	127.7	7.83	43.2	23.15
250000	225.7	4.43	58.3	17.14
500000	141.3	7.08	52.2	19.15
750000	138.7	7.21	52.0	19.24
1000000	197.4	5.07	57.9	17.29
2000000	164.9	6.07	57.2	17.51
5000000	194.1	5.15	81.2	12.32

Графики результатов замеров представлены на рисунках 3.1 и 3.2:

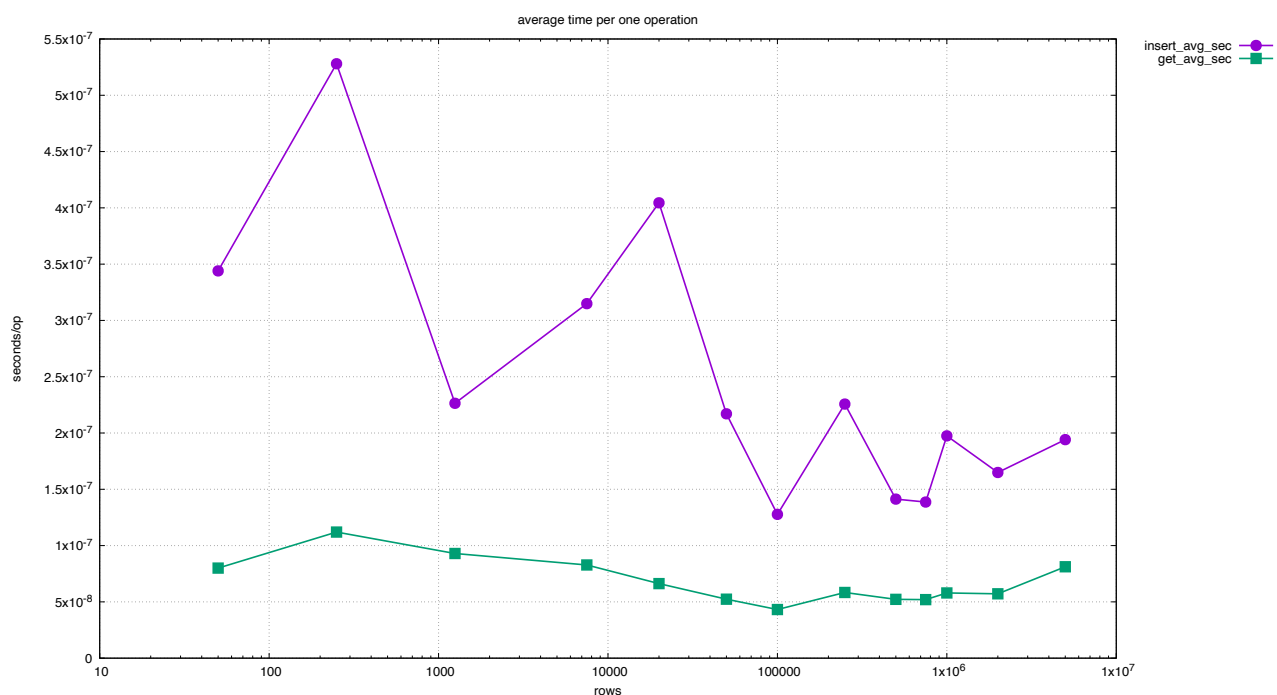


Рисунок 3.1 – Средняя задержка операций вставки/поиска для perfect hash

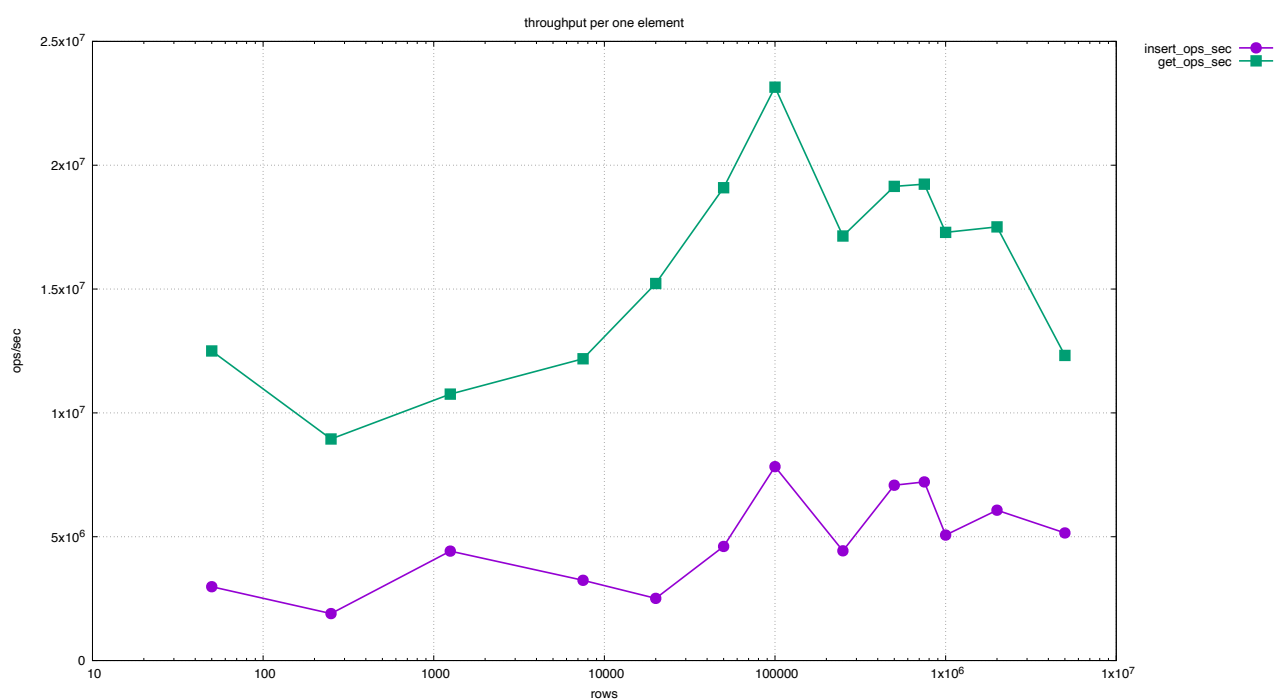


Рисунок 3.2 – Пропускная способность операций вставки/поиска для perfect hash

4 Вывод

Реализована perfect hash таблица для строковых ключей с поддержкой операций вставки и поиска. Проведены замеры производительности на наборах данных разного размера; результаты приведены в таблице и на графиках.